PROGRAMME PERSONNALISÉ — FORMATION + COACHING

Devenir développeur full-stack freelance en 12 mois

Phase intensive de 60 jours détaillée **jour par jour** (thème, concepts, ressources, exercice, objectif mesurable), puis plan mensuel de croisière sur 10 mois. Prospection client intégrée dès la semaine 6.

60 jours

6 h/jour, tous les jours. HTML → CSS → JS → React → Node → BDD → Auth → Déploiement.

7 projets

pensés pour convaincre un client, tous en ligne avec une vraie URL.

Clients tôt

portfolio en ligne au jour 28, prospection lancée au jour 41.

Profil : grand débutant · Windows (terminal Git Bash) · 6 h/jour · ressources en français en priorité
Document de référence — chaque journée terminée est validée avec ton coach avant de passer à la suivante.

Sommaire

Mode d'emploi : rythme, protocole, règles d'or 3

Semaine 1 — HTML, CSS, responsive + Projet 1 (landing page) 5

Semaine 2 — Mise en ligne + JavaScript : les fondations 8

Semaine 3 — Le DOM : JavaScript rencontre la page + Projet 2 11

Semaine 4 — JS moderne, asynchrone, API + Portfolio v1 en ligne 14

Semaine 5 — Git pro, npm + React : les fondations 17

Semaine 6 — React pro + Projet 6 + lancement de la prospection 20

Semaine 7 — Backend : Node, Express, MongoDB, full-stack 23

Semaine 8 — Authentification + Projet final full-stack 27

Semaine 9 — Finitions, portfolio v2, prospection intensive (J57–60) 30

Mois 3 à 12 — plan de croisière mensuel 32

Guide prospection freelance : clients, pitch, tarifs, statut 34

Annexes : checklist qualité projet, grille d'auto-évaluation 37

Mode d'emploi

Ce document est notre **référence commune**. Tu le suis jour par jour ; chaque soir tu me dis « Jour X terminé », je te pose 2-3 questions de vérification, et on ne passe au jour suivant que si c'est validé. Il est calibré pour ton profil : grand débutant, Windows, 6 h/jour, ressources en français en priorité.

Ta journée type (6 h)

Bloc	Durée	Contenu
1. Apprentissage actif	2 h 30	Cours, docs, vidéos du jour — en tapant **toi-même** chaque exemple. Interdiction de regarder une vidéo sans éditeur ouvert. Zéro copier-coller.
2. Pratique	3 h	L'exercice ou le projet du jour, tutoriel fermé. C'est ici que tu apprends vraiment. Si tu bloques > 25 min sur un point : note la question, contourne, avance.
3. Consolidation	30 min	Refais de mémoire une chose du matin, relis tes notes, prépare tes questions pour moi, commit Git du jour.

À partir du Jour 43, le bloc 2 passe à 2 h 30 et tu ajoutes **30 min de prospection quotidienne** (détail en semaine 6). C'est non négociable : les clients ne tombent pas du ciel le jour 60.

Le protocole de validation (rappel du contrat)

1. Tu termines une journée → tu m'écris **« Jour X terminé »** dans notre conversation.
2. Je te pose **2-3 questions de vérification** (concepts + un mini-défi de code, sans regarder tes notes).
3. Réponses solides → journée validée, on avance. Réponses floues → **on ne passe pas au jour suivant** : réexplication + exercice de rattrapage, puis nouvelle vérification.
4. Un jour raté n'est pas un drame. Trois jours enchaînés en survolant, si : le retard se paie au moment des projets.

Les règles d'or (non négociables)

- **Tape tout, ne copie rien.** La mémoire des doigts fait 50 % de l'apprentissage d'un débutant.
- **Maximum 30 % du temps en vidéo/lecture.** Le reste, c'est toi qui codes. Le « tutorial hell » (enchaîner les tutos en croyant progresser) est la première cause d'échec des autodidactes.
- **Un projet = un dépôt GitHub**, avec des commits réguliers et des messages clairs, dès la semaine 1.
- **Fini vaut mieux que parfait.** Un projet moche mais en ligne bat un projet magnifique sur ton disque dur.
- **Chaque objectif du jour se teste à voix haute.** Si tu ne peux pas l'expliquer simplement, tu ne l'as pas compris — et je le verrai aux questions de validation.
- **Anglais :** les ressources principales sont en français, mais habitue-toi progressivement à lire les docs en anglais (avec traduction automatique au début si besoin). Aucun dev pro n'y échappe.
- **Repos :** 6 h/jour tous les jours pendant 60 jours, c'est un marathon. Dors, bouge, et si un jour tu es cuit, fais 3 h de pratique plutôt que 6 h de vidéos passives.

Point d'honnêteté avant de commencer

En 60 jours tu ne seras pas senior, et personne de sérieux ne te le promettra. Tu seras capable de livrer **des landing pages, des sites vitrines et des petites applications web complètes** (React + Node + base de données), proprement et en ligne. C'est exactement le marché accessible à un débutant freelance sérieux — et c'est suffisant pour facturer tes premières centaines, puis milliers d'euros. Le reste vient avec les mois 3 à 12 et les vraies missions.

Vue d'ensemble des 60 jours

Semaine	Thème	Livrable
1 (J1–7)	Environnement, HTML, CSS, Flexbox, responsive, Git	**Projet 1** : landing page commerce local
2 (J8–14)	Mise en ligne + fondations JavaScript	Site en ligne (Netlify) + jeu console JS
3 (J15–21)	DOM, événements, formulaires, localStorage	**Projet 2** : site interactif + simulateur de devis
4 (J22–28)	JS moderne, asynchrone, fetch, API	**Projet 3** : app météo · **Projet 4** : portfolio v1 en ligne
5 (J29–35)	Git pro, npm, React (composants, props, state, effets)	**Projet 5** : app React branchée sur une API
6 (J36–42)	React Router, Tailwind + **lancement prospection**	**Projet 6** : site client complet en React · profils Malt/LinkedIn
7 (J43–49)	Node, Express, API REST, MongoDB, full-stack	API CRUD déployée + front branché dessus
8 (J50–56)	Authentification (bcrypt, JWT) + projet final	**Projet 7** : app de réservation full-stack avec espace admin
9 (J57–60)	Finitions, portfolio v2, prospection intensive	Portfolio orienté client + pipeline de 20+ prospects

SEMAINE 1 · JOURS 1–7

HTML, CSS, responsive + premier projet

Objectif de la semaine : être capable de construire seul une page web complète, propre, responsive, versionnée sur GitHub. C'est la matière première de 80 % des missions freelance de débutant.

JOUR 1 Environnement de travail + comprendre le web

CONCEPTS

Client / serveur, ce que fait un navigateur, rôles de HTML (structure), CSS (style), JS (comportement). Arborescence de fichiers. Terminal : `cd`, `ls`, `mkdir`, `touch`, `pwd`. Prise en main de VS Code et des DevTools.

INSTALLATION

VS Code (code.visualstudio.com) + extensions **Live Server**, **Prettier**, **Auto Rename Tag** · **Git for Windows** (git-scm.com), qui installe le terminal **Git Bash** — configure-le comme terminal par défaut dans VS Code · navigateur Chrome.

RESSOURCES

MDN (developer.mozilla.org, en français), section « Apprendre le développement web » : articles « Commencer avec le Web » et « Comment fonctionne Internet » · doc VS Code « Getting Started » (survol).

EXERCICE

Uniquement via Git Bash : crée l'arborescence `dev/semaine-01/jour-01`, un fichier `index.html` avec le squelette de base, ouvre-le avec Live Server, modifie le texte, observe le rechargement. Puis ouvre les DevTools (F12) sur 3 vrais sites et modifie leur contenu localement dans l'onglet Éléments.

OBJECTIF MESURABLE

Je crée un projet et j'affiche une page via Live Server en moins de 5 minutes sans aide, et j'explique à voix haute le trajet navigateur → serveur → réponse.

JOUR 2 HTML : la structure

CONCEPTS

Anatomie d'un élément (balise, attribut, contenu), squelette complet (`doctype`, `html`, `head`, `meta`, `body`), titres h1–h6, paragraphes, listes `ul` / `ol`, liens (relatifs vs absolus), images (`src`, `alt`), commentaires.

RESSOURCES

MDN, module « Introduction au HTML » (FR) · Grafikart.fr, formation gratuite **« Apprendre le HTML »** (chapitres structure, liens, images) · validator.w3.org pour vérifier ton code.

EXERCICE

Page « À propos de moi » : photo, bio, liste de tes objectifs, liens vers 3 sites que tu aimes — **zéro erreur au validateur W3C**.

OBJECTIF MESURABLE

J'écris le squelette HTML de mémoire et je sais expliquer à quoi sert `alt` et la différence lien relatif / absolu.

JOUR 3 HTML sémantique, formulaires, tableaux

CONCEPTS

`header`, `nav`, `main`, `section`, `article`, `footer` · `div` vs balises sémantiques et pourquoi (SEO + accessibilité — argument que tu revendras à tes clients) · formulaires : `label` + `for`, types d'`input`, `textarea`, `select`, `button` · tableaux · bases d'accessibilité.

RESSOURCES

MDN : « Structurer le web » et le module « Formulaires HTML » (premiers chapitres, FR) · Grafikart, formation HTML (chapitre formulaires).

EXERCICE

Page « Contact » d'une entreprise fictive : formulaire complet (nom, email, téléphone, sujet en menu déroulant, message, case à cocher RGPD) + tableau d'horaires d'ouverture. Chaque champ a son label. 100 % sémantique, validé W3C.

OBJECTIF MESURABLE

Je code un formulaire accessible sans modèle et je justifie chaque balise sémantique choisie.

JOUR 4 CSS : sélecteurs et box model

CONCEPTS

Fichier CSS externe (et pourquoi), sélecteurs (type, `.classe`, `#id`, descendant), notion de spécificité, typographie (`font-family`, `font-size`, `line-height`), couleurs (hex, rgb), **le box model** (content / padding / border / margin), `box-sizing: border-box`, `display` block vs inline.

RESSOURCES

MDN : « Premiers pas en CSS » + article « Le modèle de boîte » (FR) · Grafikart, formation gratuite **« Apprendre le CSS »** (début) · DevTools, onglet Éléments, pour visualiser les boîtes.

EXERCICE

Stylise entièrement ta page du Jour 2. Puis crée 3 « cartes » avec padding, bordure et marges différents, inspecte-les et corrige-les dans DevTools jusqu'à obtenir des espacements identiques.

OBJECTIF MESURABLE

Je dessine le box model sur papier de mémoire et je débogue un problème d'espacement avec DevTools en expliquant margin vs padding.

JOUR 5 Flexbox — la journée la plus rentable de ta semaine

CONCEPTS

`display:flex`, axe principal / secondaire, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex-wrap`, centrage vertical + horizontal.

RESSOURCES

Flexbox Froggy (flexboxfroggy.com, disponible en français) : les 24 niveaux · CSS-Tricks « A Complete Guide to Flexbox » à mettre en favori (anglais, très visuel) · Grafikart, formation CSS, chapitre flexbox.

EXERCICE

Sans rien copier : 1) une navbar (logo à gauche, liens à droite) · 2) une rangée de 3 cartes tarifs de largeur égale · 3) un footer 3 colonnes · 4) un élément parfaitement centré au milieu de la page.

OBJECTIF MESURABLE

Je centre n'importe quoi de tête et je code une navbar en moins de 15 minutes.

JOUR 6 Responsive + Grid (survol) + premiers commits Git

CONCEPTS

Meta viewport, unités relatives (`rem` , %), media queries `min-width` , approche **mobile-first**, `max-width:100%` sur les images · Grid basique : `grid-template-columns` , `repeat` , `fr` , `gap` · quand Grid vs Flexbox · mode appareil des DevTools. En fin de journée (1 h) : `git init` , `git status` , `git add` , `git commit` .

RESSOURCES

MDN « Media queries » (FR) · **Grid Garden** (cssgridgarden.com, FR) · CSS-Tricks « Complete Guide to Grid » en favori · Git : formation gratuite Grafikart **« Apprendre Git »** (3 premières vidéos) ou le livre **Pro Git**, chapitres 1–2, gratuit en français sur git-scm.com.

EXERCICE

Rends responsives tes pages des jours 2–5 (mobile → desktop) · galerie de 6 images en Grid qui passe de 3 colonnes à 2 puis 1 selon l'écran · versionne toute ta semaine avec des commits propres.

OBJECTIF MESURABLE

Aucune de mes pages n'a de défilement horizontal en mode iPhone dans DevTools ; je sais faire init/add/commit et expliquer ce qu'est un commit.

JOUR 7 PROJET 1 — Landing page complète + GitHub + bilan

PROJET (4 H 30)

Landing page one-page pour un **commerce local fictif** — choisis un vrai type de client freelance (restaurant, photographe, coach sportif). Sections : hero avec titre + bouton d'appel à l'action, 3 services en cartes, à propos, témoignages, formulaire de contact, footer. Mobile-first, sémantique, zéro framework, zéro copier-coller. Pour le design, inspire-toi d'un challenge gratuit sur **frontendmentor.io** ou de vrais sites du secteur.

GITHUB (1 H)

Crée ton compte GitHub, crée un dépôt, pousse ton projet en suivant le guide officiel « Hello World » de la doc GitHub.

BILAN (30 MIN)

Auto-évaluation avec la checklist qualité en annexe, puis message « Jour 7 terminé » à ton coach.

OBJECTIF MESURABLE

Une page complète, responsive, codée seul en une journée, publique sur GitHub. C'est la v1 de ta pièce de portfolio n°1 — elle sera en ligne avec une vraie URL dès le Jour 8.

AUTO-CONTRÔLE DE FIN DE SEMAINE 1

Réponds à voix haute, sans notes. Si une réponse est floue, la journée correspondante n'est pas acquise :

- Quelle est la différence entre `margin` et `padding` ? Entre une classe et un id ?
- Comment centres-tu un élément horizontalement ET verticalement avec Flexbox ?
- Pourquoi utiliser `<nav>` plutôt que `<div class="nav">` ? Donne deux raisons.
- Qu'est-ce qu'un commit ? Que font `git add` puis `git commit` ?
- Défi : recoder une navbar responsive de mémoire en moins de 20 minutes.

SEMAINE 2 · JOURS 8–14

Mise en ligne + JavaScript : les fondations

Ton projet 1 obtient une vraie URL dès lundi. Puis on attaque LE gros morceau : JavaScript. Cette semaine, tout se passe dans la console — pas de page web. C'est volontaire : on isole la logique pure avant de la brancher sur le HTML.

JOUR 8 Déploiement Netlify + premiers pas JavaScript

CONCEPTS

Hébergement statique, ce qu'est un déploiement, Netlify (drag & drop puis connexion au dépôt GitHub = déploiement continu) · JS : balise `<script>`, la console, `console.log`, variables `let` / `const`, types primitifs (string, number, boolean, null, undefined), template literals `` `${...}` ``.

RESSOURCES

Doc Netlify « Get started » (guidée, 15 min) · Grafikart.fr, formation gratuite **« Apprendre JavaScript »** (chapitres variables et types) · MDN, « Premiers pas en JavaScript » (FR).

EXERCICE

1) Déploie le Projet 1 sur Netlify via GitHub : modifie une ligne, pousse, vérifie que le site se met à jour seul. 2) Console : déclare des variables de chaque type, affiche des phrases dynamiques avec les template literals, provoque volontairement une erreur de réassignation d'une `const` et lis le message.

OBJECTIF MESURABLE

Mon projet a une URL publique que je peux envoyer à n'importe qui ; je sais expliquer let vs const et nommer les types primitifs.

JOUR 9 Opérateurs et conditions

CONCEPTS

Opérateurs arithmétiques et de comparaison — **la différence `==` vs `===`** · opérateurs logiques `&&`, `||`, `!` · `if / else if / else`, opérateur ternaire, `switch` · valeurs truthy / falsy.

RESSOURCES

Grafikart JS (chapitre conditions) · MDN « Prendre des décisions dans le code » · fr.javascript.info (chapitres opérateurs et comparaisons — traduction française du meilleur manuel JS du web).

EXERCICE

Mini-programmes console : pair/impair · majeur/mineur à partir d'une année de naissance · le classique **FizzBuzz** (multiples de 3 → « Fizz », de 5 → « Buzz », des deux → « FizzBuzz ») · simulateur de tarif : si surface > 100 m² alors prix majoré, etc.

OBJECTIF MESURABLE

J'explique == vs === avec un exemple piège, et j'écris FizzBuzz sans aide en moins de 10 minutes.

JOUR 10 Boucles

CONCEPTS

`for`, `while`, `for...of`, `break` / `continue`, boucles imbriquées, compteurs et accumulateurs.

RESSOURCES

Grafikart JS (chapitre boucles) · MDN « Les boucles dans le code » (FR).

EXERCICE

Table de multiplication complète · somme des nombres de 1 à 100 · pyramide d'étoiles (boucles imbriquées) · jeu « devine le nombre » avec `prompt()` et compteur d'essais.

OBJECTIF MESURABLE

Je choisis la bonne boucle selon la situation et j'écris une boucle imbriquée en expliquant qui tourne dans qui.

JOUR 11 Fonctions

CONCEPTS

Déclaration vs expression de fonction, paramètres / arguments, `return` (et la différence avec `console.log` !), portée des variables (scope), fonctions fléchées `=>`, paramètres par défaut.

RESSOURCES

Grafikart JS (chapitre fonctions) · MDN « Fonctions — réutiliser du code » · fr.javascript.info (fonctions, fonctions fléchées).

EXERCICE

Refactore les exercices des jours 9–10 en fonctions réutilisables · crée ta « bibliothèque » de 5 fonctions utilitaires : `calculTTC(prixHT)`, `estMajeur(age)`, `formatPrix(nombre)`, `genererPseudo(prenom, annee)`, `max(a, b)`.

OBJECTIF MESURABLE

J'écris une fonction fléchée avec valeur de retour, et j'explique pourquoi une variable déclarée dans une fonction n'existe pas dehors.

JOUR 12 Tableaux

CONCEPTS

Création, index (à partir de 0 !), `length`, `push` / `pop` / `shift` / `unshift`, `indexOf` / `includes`, `slice` vs `splice`, parcours avec `for...of` et `forEach`.

RESSOURCES

Grafikart JS (chapitre tableaux) · MDN « Les tableaux » (FR) · fr.javascript.info (tableaux, méthodes de tableaux — première partie).

EXERCICE

Gestionnaire de courses en console : fonctions `ajouter`, `retirer`, `afficher` · trouver le min et le max d'un tableau de prix sans `Math.min` · inverser un tableau sans `.reverse()`.

OBJECTIF MESURABLE

Je fais un CRUD en mémoire (créer, lire, modifier, supprimer) sur un tableau sans regarder la doc pour les méthodes de base.

JOUR 13 Objets

CONCEPTS

Propriétés et méthodes, notation point vs crochets, objets imbriqués, **tableaux d'objets** (la structure reine du web : listes de produits, de clients, de réservations…), survol de `this`, introduction au format JSON.

RESSOURCES

Grafikart JS (chapitre objets) · MDN « Les bases des objets JavaScript » (FR).

EXERCICE

Modélise des clients : tableau d'objets `{nom, email, budget}` · fonction qui filtre les clients dont le budget dépasse un seuil · fonction qui affiche une fiche formatée par client · accède à une propriété imbriquée (`client.adresse.ville`).

OBJECTIF MESURABLE

Je crée et parcours un tableau d'objets, et j'accède à n'importe quelle propriété, même imbriquée.

JOUR 14 PROJET console + première dose d'entraînement

PROJET (4 H)

Au choix : **« Le Juste Prix »** (nombre aléatoire avec `Math.random`, `prompt`, indices « plus / moins », compteur d'essais, proposition de rejouer) ou **quiz à choix multiples** avec score final et messages selon le résultat. En pur JS, découpé en fonctions propres, sans tutoriel.

ENTRAÎNEMENT (1 H 30)

Crée un compte sur **Codewars** et résous 3 katas de niveau 8 kyu (le plus facile). C'est ta salle de sport JS : tu y retourneras régulièrement.

BILAN (30 MIN)

Pousse tout sur GitHub, note ce qui a bloqué, message « Jour 14 terminé ».

OBJECTIF MESURABLE

Un programme complet écrit seul (logique + fonctions + boucles + conditions), sur GitHub. Si tu y arrives sans tutoriel, tes fondations JS tiennent.

AUTO-CONTRÔLE DE FIN DE SEMAINE 2

- Différence entre `return` et `console.log` ? Entre `slice` et `splice` ?
- Que vaut `"5" == 5` ? Et `"5" === 5` ? Pourquoi ?
- Défi : écrire une fonction qui reçoit un tableau d'objets `{nom, prix}` et retourne le nom du produit le plus cher — sans aide, en 15 minutes.
- Explique à voix haute ce qu'est le déploiement continu que tu as mis en place au Jour 8.

SEMAINE 3 · JOURS 15–21

Le DOM : JavaScript rencontre la page

Cette semaine, ton JS sort de la console et anime de vraies pages. À la fin : ton Projet 1 devient un site interactif digne d'être vendu, et tu construis un simulateur de devis — l'outil qui impressionne le plus les petits clients.

JOUR 15 DOM : sélectionner et manipuler la page

CONCEPTS

Ce qu'est le DOM (l'arbre de la page vue par JS) · `querySelector` / `querySelectorAll` · `textContent` vs `innerHTML` (et le risque de sécurité du second) · modifier styles et classes avec `classList` (add / remove / toggle) · `createElement`, `append`, `remove`.

RESSOURCES

Grafikart JS (chapitres sur le DOM) · MDN « Manipuler des documents » (FR).

EXERCICE

Page « annuaire d'équipe » : à partir de ton tableau d'objets clients du Jour 13, génère une carte HTML par personne, entièrement depuis JavaScript. Bonus : un bouton « ajouter un membre » qui crée une carte supplémentaire.

OBJECTIF MESURABLE

Je génère du HTML dynamiquement à partir d'un tableau d'objets — le mécanisme au cœur de toutes les applis web.

JOUR 16 Événements

CONCEPTS

`addEventListener`, l'objet `event`, événements `click`, `input`, `submit`, `keydown` · `preventDefault` · bases de la délégation d'événements (écouter le parent).

RESSOURCES

Grafikart JS (chapitre événements) · MDN « Introduction aux événements » (FR).

EXERCICE

Compteur +/− avec remise à zéro · champ texte qui affiche la saisie en direct · et surtout : **un menu burger fonctionnel sur ta landing page du Jour 7** (ouverture/fermeture au clic, en mobile).

OBJECTIF MESURABLE

Ma landing page a un menu mobile qui fonctionne, codé par moi, déjà en ligne grâce au déploiement continu.

JOUR 17 Formulaires + validation JavaScript

CONCEPTS

Récupérer les valeurs des champs, événement `submit` + `preventDefault`, validation : champs vides, longueur minimale, format email (regex simple fournie, pas à inventer) · afficher / retirer des messages d'erreur sous les champs · bonnes pratiques UX de validation.

RESSOURCES

MDN « Validation des données de formulaires » (FR) · Grafikart JS.

EXERCICE

Branche une validation complète sur le formulaire de contact du Projet 1 : erreurs en rouge sous chaque champ concerné, message de succès si tout est bon, focus sur le premier champ en erreur.

OBJECTIF MESURABLE

Mon formulaire bloque l'envoi si les données sont invalides et guide l'utilisateur — comme sur un vrai site pro.

JOUR 18 Méthodes de tableaux niveau pro : map, filter, reduce

CONCEPTS

`map`, `filter`, `find`, `some` / `every`, `sort`, introduction à `reduce`, chaînage de méthodes. **Journée critique : c'est le cœur de React** que tu verras en semaine 5 — tout affichage de liste en React passe par `map`.

RESSOURCES

fr.javascript.info (méthodes de tableaux) · MDN (fiches map / filter / reduce, FR) · Grafikart a des vidéos dédiées aux méthodes de tableaux — cherche « tableaux » sur grafikart.fr.

EXERCICE

Sur un tableau de 10 produits `{nom, prix, stock}` : liste des noms (map) · produits à moins de 50 € (filter) · total du panier (reduce) · tri par prix croissant (sort) · puis affiche chaque résultat dans le DOM sous forme de cartes.

OBJECTIF MESURABLE

J'utilise map / filter / reduce sans doc et je sais dire ce que chacune retourne (nouveau tableau, tableau filtré, valeur unique).

JOUR 19 localStorage + JSON : faire survivre les données

CONCEPTS

`JSON.stringify` / `JSON.parse`, `localStorage` (setItem / getItem / removeItem), persistance côté navigateur et ses limites, pattern « charger au démarrage, sauver à chaque changement ».

RESSOURCES

MDN « Window.localStorage » (FR) · Grafikart JS.

EXERCICE

1) Mode sombre avec préférence mémorisée (la page se recharge dans le bon thème). 2) Liste de tâches persistante : ajout, suppression, coche, tout survit au rafraîchissement.

OBJECTIF MESURABLE

Mes données survivent au rafraîchissement de la page, et je sais expliquer pourquoi il faut stringify avant de stocker.

JOURS 20–21 PROJET 2 — Site interactif pro + simulateur de devis

JOUR 20 — LE PACK INTERACTIVITÉ

Transforme la landing page du Projet 1 en site « niveau agence » : menu burger (déjà fait), **FAQ en accordéon**, **carrousel de témoignages** simple (boutons précédent/suivant), validation de formulaire (déjà faite), **mode sombre persistant**, apparition des sections au scroll (via une classe CSS ajoutée avec IntersectionObserver — je te guiderai si besoin).

JOUR 21 — LE SIMULATEUR DE DEVIS

Page « Estimez votre projet » pour le commerce fictif : l'utilisateur coche des options (ex. pour un photographe : type de séance, durée, retouches, déplacement), **le prix se calcule et s'affiche en direct**, avec un récapitulatif et un bouton « envoyer ma demande ». Données des options dans un tableau d'objets, calcul avec reduce.

RESSOURCES

Aucun tutoriel pour ce projet : tes notes, MDN, et moi. C'est voulu.

OBJECTIF MESURABLE

Deux briques ultra-vendables à un client (site interactif + simulateur), en ligne sur Netlify, sur GitHub. Un artisan qui voit un simulateur de devis sur ton portfolio comprend immédiatement ta valeur.

AUTO-CONTRÔLE DE FIN DE SEMAINE 3

- Que retourne `filter` ? Et `reduce` ? Donne un exemple d'usage réel de chacun.
- Pourquoi `preventDefault()` sur un submit ? Que se passe-t-il sans ?
- Quelle est la différence entre `textContent` et `innerHTML`, et quel risque pose le second ?
- Défi : de mémoire, afficher dans le DOM la liste des produits en stock triés par prix, à partir d'un tableau d'objets (20 min max).

SEMAINE 4 · JOURS 22–28

JS moderne, asynchrone, API + Portfolio v1 en ligne

Tu apprends à parler aux serveurs (fetch, API, async/await) — la compétence qui sépare « je fais des pages » de « je fais des applications ». Et vendredi, ton portfolio est en ligne : à partir du Jour 28, tu es techniquement démarchable.

JOUR 22 JavaScript moderne (ES6+)

CONCEPTS

Destructuring d'objets et de tableaux, opérateurs spread / rest `...`, raccourcis de propriétés, modules `import` / `export` (avec `<script type="module">`), rappel fonctions fléchées. Toute la syntaxe que React utilise en permanence.

RESSOURCES

fr.javascript.info (destructuring, modules) · Grafikart : cherche « ES6 » / « nouveautés JavaScript » sur grafikart.fr · MDN (fiches destructuring et modules, FR).

EXERCICE

Refactore le simulateur de devis en 3 modules : `data.js` (les options), `ui.js` (affichage), `main.js` (logique) · utilise destructuring et spread partout où c'est pertinent · copie un objet avec spread et prouve que l'original n'est pas modifié.

OBJECTIF MESURABLE

Je découpe un projet en modules et je lis sans effort une ligne comme `const {nom, prix} = produit`.

JOUR 23 Asynchrone : comprendre les promesses

CONCEPTS

Synchrone vs asynchrone (pourquoi le navigateur ne peut pas attendre), `setTimeout`, callbacks et leurs limites, **Promise** : états (pending / fulfilled / rejected), `.then` / `.catch` / `.finally`.

RESSOURCES

Grafikart : vidéo « Comprendre les Promesses » (cherche « promesses » sur grafikart.fr) · MDN « Utiliser les promesses » (FR) · fr.javascript.info (chapitre promesses).

EXERCICE

Crée une fausse API : fonction `chargerProduits()` qui retourne une Promise résolue après 2 secondes (setTimeout) · affiche « chargement… » puis les données · enchaîne deux opérations avec .then · fais échouer la promesse et gère l'erreur avec .catch.

OBJECTIF MESURABLE

J'explique ce qu'est une promesse avec mes mots et je lis du code .then/.catch sans paniquer.

JOUR 24 fetch + async/await : consommer une API

CONCEPTS

Ce qu'est une API REST, requête HTTP GET, réponse JSON · `fetch`, `response.json()` · **async / await** + `try / catch` · vérifier `response.ok` · afficher un état de chargement et une erreur propre.

RESSOURCES

MDN « Utiliser l'API Fetch » (FR) · Grafikart (cherche « fetch ») · APIs publiques gratuites pour t'entraîner : **PokeAPI** (pokeapi.co), **TheMealDB** (themealdb.com), **API Adresse** de data.gouv.fr (française, sans clé).

EXERCICE

Page qui charge et affiche 12 recettes de TheMealDB (ou 12 Pokémon) en cartes, avec un loader pendant le chargement et un message d'erreur si le réseau échoue (coupe ton wifi pour tester !). Version .then le matin, version async/await l'après-midi : compare.

OBJECTIF MESURABLE

Je consomme n'importe quelle API REST publique avec async/await + try/catch, avec gestion du chargement et des erreurs.

JOUR 25 PROJET 3 — Application météo

PROJET (JOURNÉE COMPLÈTE)

App météo avec **OpenWeatherMap** (clé API gratuite) : champ de recherche de ville, affichage température / ressenti / icône / description en français (paramètres `lang=fr&units=metric`), gestion de la ville inconnue (erreur 404 → message clair), design soigné et responsive. Bonus si le temps le permet : météo de la position de l'utilisateur via la géolocalisation du navigateur.

RESSOURCES

Doc OpenWeatherMap « Current weather data » · tes acquis des jours 23–24. Pas de tutoriel « weather app » sur YouTube : tu construis, je débloque.

OBJECTIF MESURABLE

Une application API complète, en ligne sur Netlify, dans le portfolio. Attention : ne mets jamais une clé API sensible en public — pour OpenWeather gratuit c'est toléré, on verra la vraie solution (backend) en semaine 7.

JOUR 26 Déboguer comme un pro + qualité de code

CONCEPTS

Lire un message d'erreur (type, ligne, pile d'appels), les erreurs classiques (`undefined is not a function`, `cannot read property of undefined`) · **breakpoints dans DevTools** (onglet Sources) : pause, avance pas à pas, inspection des variables · `console.table`, `console.error` · Prettier configuré pour formater à la sauvegarde · conventions de nommage et structure de projet propre.

RESSOURCES

Doc Chrome DevTools, section « Debug JavaScript » (anglais, très visuel — bon exercice de lecture EN) · MDN « Qu'est-ce qui a mal tourné ? Déboguer JavaScript » (FR).

EXERCICE

Reprends ton app météo et ton simulateur : introduis volontairement 5 bugs (faute de frappe dans une propriété, oubli d'await, mauvais sélecteur…), puis corrige-les **uniquement avec les breakpoints**, sans console.log. Ensuite, nettoie le code des 3 projets : noms clairs, fonctions courtes, zéro code mort.

OBJECTIF MESURABLE

Je débogue avec des breakpoints et je lis un message d'erreur au lieu de le subir.

JOURS 27–28

PROJET 4 — Portfolio v1 : tu deviens démarchable

CONTENU (J27)

Structure : hero avec **proposition de valeur orientée client** (« Sites web modernes et rapides pour commerçants et indépendants » — pas « développeur junior passionné ») · 3 projets présentés en **mini-études de cas** : contexte → solution → résultat, avec lien live + lien GitHub · section à propos (courte, orientée fiabilité) · formulaire de contact branché sur **Netlify Forms** (tu reçois les messages par email sans backend — lis la doc Netlify Forms, 10 min).

FINITIONS (J28)

SEO de base : `title` unique, `meta description`, balises Open Graph · images compressées (squoosh.app) · favicon · test Lighthouse (DevTools, onglet Lighthouse) : vise > 90 en performance et accessibilité · relecture orthographe impitoyable. Option : nom de domaine (~10 €/an chez OVH ou Namecheap) branché sur Netlify.

OBJECTIF MESURABLE

Une URL de portfolio professionnelle prête à être envoyée. À partir de ce soir, tu peux techniquement répondre à une annonce « création de site vitrine ». On lance la prospection organisée en semaine 6, mais si une occasion se présente avant : fonce, je t'aiderai.

AUTO-CONTRÔLE DE FIN DE SEMAINE 4

- Explique async/await à quelqu'un qui ne code pas, en 3 phrases.
- Que fait `try/catch` et pourquoi est-ce indispensable avec fetch ?
- Défi : charger et afficher les données d'une API que tu n'as jamais utilisée (je t'en donnerai une) en moins de 30 minutes.
- Montre-moi ton score Lighthouse. En dessous de 85, on corrige avant d'avancer.

SEMAINE 5 · JOURS 29–35

Git pro, npm + React : les fondations

React est la bibliothèque front la plus demandée du marché freelance. Bonne nouvelle : tout ce que tu as appris (fonctions, tableaux d'objets, map, destructuring, modules) est exactement ce que React manipule. La doc officielle react.dev existe en français et elle est excellente — ce sera ta ressource principale.

JOUR 29 Git niveau pro + npm

CONCEPTS

Branches : `git branch`, `git switch`, `git merge` · résoudre un conflit (en provoquer un exprès pour dédramatiser) · `.gitignore` · écrire un vrai README (description, capture, techno, lien live) · côté outillage : installer **Node.js LTS** (nodejs.org), comprendre `npm install`, `node_modules`, `package.json`, les scripts npm.

RESSOURCES

Grafikart « Apprendre Git » (suite : branches et fusion) · livre Pro Git, chapitre 3 (FR, git-scm.com) · doc npm « Getting started » (survol).

EXERCICE

Sur le portfolio : crée une branche `feature/nouvelle-section`, modifie, fusionne dans main · provoque un conflit (modifie la même ligne dans deux branches) et résous-le calmement · écris un README digne de ce nom pour tes 4 projets.

OBJECTIF MESURABLE

Je travaille en branches, je résous un conflit sans paniquer, et j'explique ce que contient package.json.

JOUR 30 React : premier contact

CONCEPTS

Pourquoi React (interface = fonction des données, composants réutilisables) · créer un projet avec **Vite** (`npm create vite@latest`) · structure des fichiers, serveur de dev · **JSX** : une seule racine, `className`, expressions entre `{ }` · écrire et imbriquer ses premiers composants.

RESSOURCES

fr.react.dev, section « Apprendre » : « Démarrage rapide » puis « Votre premier composant » et « Écrire du balisage avec JSX » · doc Vite (vitejs.dev, guide de démarrage) · Grafikart « Apprendre React » en complément vidéo.

EXERCICE

Crée un projet Vite + React · recrée le header, le hero et le footer de ton portfolio en composants séparés (`Header.jsx`, `Hero.jsx`, `Footer.jsx`) assemblés dans `App.jsx`.

OBJECTIF MESURABLE

Je crée un projet React from scratch et j'explique ce qu'est un composant et les 3 règles de base du JSX.

JOUR 31 Props + rendu de listes

CONCEPTS

Passer des données du parent à l'enfant avec les **props** (avec destructuring !) · composition · rendu conditionnel (`&&`, ternaire) · afficher des listes avec `map` et l'attribut `key` (pourquoi il est obligatoire) · découper une maquette en composants.

RESSOURCES

fr.react.dev, chapitres « Passer des props à un composant », « Affichage conditionnel », « Afficher des listes ».

EXERCICE

Composant `CarteProjet` réutilisable (props : titre, description, image, lien) alimenté par un tableau d'objets · liste de produits avec badge « rupture de stock » affiché conditionnellement · composant `Badge` dont la couleur dépend d'une prop.

OBJECTIF MESURABLE

Je fais circuler des données parent → enfant et j'affiche proprement une liste depuis un tableau d'objets (avec key).

JOUR 32 State et événements : l'interactivité React

CONCEPTS

`useState` : lire, mettre à jour, re-rendu automatique · **l'immutabilité** (pourquoi on ne modifie jamais le state directement, surtout tableaux et objets : on crée une copie avec spread) · gérer les événements `onClick`, `onChange` · plusieurs états dans un composant.

RESSOURCES

fr.react.dev : « L'état : la mémoire d'un composant », « Répondre aux événements », « Mettre à jour les tableaux de l'état ».

EXERCICE

Compteur avec +/−/reset · interrupteur de thème sombre version React · **galerie filtrable** : boutons de catégories (Tous / Sites / Apps) qui filtrent une liste de projets (state + filter + map — le combo que tu utiliseras toute ta carrière).

OBJECTIF MESURABLE

J'explique pourquoi on ne fait jamais `tableau.push` dans le state, et je code un filtre interactif seul.

JOUR 33 Formulaires React + faire remonter l'état

CONCEPTS

Inputs contrôlés (value + onChange), formulaire multi-champs avec un objet de state · **lifting state up** : remonter l'état dans le parent commun · communication enfant → parent en passant une fonction dans les props.

RESSOURCES

fr.react.dev : « Réagir aux saisies avec l'état », « Partager l'état entre des composants ».

EXERCICE

Refais le simulateur de devis en React : composants `Options`, `Recapitulatif`, `Total`, état dans `App`. Compare avec ta version vanilla du Jour 21 : même résultat, deux philosophies. C'est l'exercice qui fait « cliquer » React chez la plupart des débutants.

OBJECTIF MESURABLE

Je fais remonter des données d'un enfant vers un parent, et je sais dire où doit vivre un état partagé.

JOUR 34 useEffect + charger des données d'API dans React

CONCEPTS

`useEffect` : à quoi ça sert, le tableau de dépendances (`[]` = au montage), pièges classiques (boucle infinie) · pattern complet de chargement : 3 états `data` / `loading` / `error` · survol honnête : « vous n'avez peut-être pas besoin d'un effet » (fr.react.dev) pour ne pas en mettre partout.

RESSOURCES

fr.react.dev : « Synchroniser grâce aux Effets » (+ survol de « Vous n'avez peut-être pas besoin d'un Effet ») · Grafikart React (vidéos useEffect).

EXERCICE

Refais l'app météo en React : recherche de ville, loader, gestion d'erreur, le tout avec le pattern data/loading/error. Encore une fois : comparer avec ta version vanilla est le meilleur cours de React qui existe.

OBJECTIF MESURABLE

Je charge des données d'API dans un composant proprement, et j'explique ce que fait le tableau de dépendances.

JOUR 35 PROJET 5 — Première app React complète

PROJET (JOURNÉE COMPLÈTE)

« Trouve ta recette » avec l'API TheMealDB : barre de recherche, grille de cartes résultats, détail d'une recette au clic (dans une modale ou une vue dédiée), **favoris sauvegardés en localStorage**. Stack : Vite + React, ton CSS.

DÉPLOIEMENT

Mets l'app en ligne sur **Vercel** (vercel.com, connexion GitHub, deux clics — lis leur guide de démarrage) : tu connaîtras ainsi les deux hébergeurs stars, Netlify et Vercel.

OBJECTIF MESURABLE

Première application React complète, en ligne, construite sans tutoriel. State, props, useEffect, fetch : tout y passe.

AUTO-CONTRÔLE DE FIN DE SEMAINE 5

- Que se passe-t-il quand on appelle la fonction de mise à jour d'un useState ?
- Pourquoi `key` est-il obligatoire dans une liste ? Pourquoi pas l'index du tableau (dans l'idéal) ?
- Comment un composant enfant peut-il « prévenir » son parent qu'un bouton a été cliqué ?
- Défi : composant compteur de caractères pour un textarea (limite 200, compteur en rouge au-delà) — 20 minutes, de mémoire.

SEMAINE 6 · JOURS 36–42

React pro + Projet 6 + lancement de la prospection

La semaine charnière : tu termines ta pièce maîtresse frontend (un site client complet en React), et le week-end, on lance officiellement la machine commerciale. À partir d'ici, tu n'es plus seulement en formation — tu es en pré-lancement d'activité.

JOUR 36 React Router : des vraies pages

CONCEPTS

SPA vs site multi-pages · `react-router-dom` : `Routes`, `Route`, `Link` / `NavLink` · routes dynamiques avec `useParams` (`/recette/:id`) · page 404 · layout partagé avec `Outlet` (navbar/footer communs).

RESSOURCES

Doc officielle React Router (reactrouter.com — anglais, suis le tutoriel de démarrage) · Grafikart a des vidéos React Router en français si besoin.

EXERCICE

Transforme l'app recettes du Jour 35 en multi-pages : `/` (recherche), `/recette/:id` (détail), `/favoris`, page 404 stylée, navbar avec lien actif souligné (NavLink).

OBJECTIF MESURABLE

Je structure une application multi-pages avec routes dynamiques, et l'URL change sans rechargement de page.

JOUR 37 Tailwind CSS : styler à la vitesse des agences

CONCEPTS

Philosophie utility-first, installation avec Vite (suis la doc officielle à la lettre), classes de base (spacing, couleurs, typo, flex/grid), responsive avec les préfixes `sm:` `md:` `lg:`, états `hover:` `focus:`, quand extraire un composant React plutôt que répéter les classes.

RESSOURCES

Doc officielle Tailwind (tailwindcss.com — guide d'installation « Vite » puis navigation dans les utilitaires ; leur moteur de recherche est excellent) · sur YouTube en français, Benjamin Code et Melvynx ont de bons contenus Tailwind/React pour la culture.

EXERCICE

Restyle entièrement l'app recettes en Tailwind : navbar, grille de cartes, page détail, boutons avec états hover, responsive complet. Objectif : ne plus écrire une seule ligne de CSS custom.

OBJECTIF MESURABLE

Je maquette une section responsive en Tailwind sans ouvrir de fichier CSS, plus vite qu'en CSS classique.

JOURS 38–40 PROJET 6 — Site client complet en React (3 jours)

LE BRIEF (COMME UN VRAI CLIENT)

Le restaurant fictif **« Chez Lucie »** (ou l'univers de ton choix — garde-le, il resservira au projet final) veut un site moderne : accueil, carte du restaurant (données dans un fichier JSON, filtrables par catégorie : entrées / plats / desserts), galerie photos, page réservation (formulaire validé), contact avec carte, mentions légales. Stack : **Vite + React + React Router + Tailwind**.

JOUR 38

Setup, structure des dossiers, layout (navbar responsive + footer), routing complet, page d'accueil avec hero et sections.

JOUR 39

Pages métier : carte filtrable (state + filter + map), galerie, formulaire de réservation avec validation et message de confirmation.

JOUR 40

Finitions : responsive parfait, images optimisées, SEO de base, **Lighthouse > 90**, mise en ligne sur Vercel, README en mini-étude de cas, ajout au portfolio.

OBJECTIF MESURABLE

LA pièce maîtresse frontend du portfolio : un site que tu peux montrer à un restaurateur en disant « je vous fais le même, adapté à vous ». Niveau de facturation visé : 800–1 500 €.

JOUR 41 PROSPECTION 1/2 — Positionnement, profils, pitch

MATIN — POSITIONNEMENT

Un débutant ne vend pas « du développement full-stack » : il vend un **résultat à une cible précise**. Choisis ta niche de départ : commerçants et indépendants locaux (restaurants, artisans, coachs, photographes, professions libérales) qui ont un site daté ou pas de site. Formule ton pitch en 3 lignes : qui tu aides + à obtenir quoi + preuve (lien portfolio). Écris 3 variantes.

APRÈS-MIDI — ÊTRE TROUVABLE

1) Profil **Malt** complet (photo pro, titre orienté client, description, 3 projets avec visuels, tarif journalier — voir grille tarifs) ; lis leurs guides « bien remplir son profil ». 2) Profil **LinkedIn** : même logique, bannière avec ta proposition de valeur, section « sélection » avec tes liens. 3) Compte **Codeur.com** (plateforme française très active sur les petits projets — parfaite pour débuter). 4) Ajoute le site restaurant au portfolio.

OBJECTIF MESURABLE

Trois profils publiés et complets, un pitch écrit en 3 variantes. N'importe qui qui te cherche te trouve, et comprend en 10 secondes ce que tu vends.

JOUR 42 PROSPECTION 2/2 — Premiers contacts + statut micro-entreprise

MATIN — LA LISTE ET LES PREMIERS MESSAGES

Construis une liste de **20 prospects locaux** via Google Maps : commerces bien notés mais sans site, ou avec un site visiblement daté / pas responsive. Pour chacun : nom, contact, problème constaté. Envoie tes **5 premiers messages personnalisés** (email ou formulaire) : un constat précis sur LEUR présence en ligne + une proposition simple (ex. maquette d'accueil gratuite ou mini-audit offert) + ton portfolio. Réponds aussi à 2 annonces sur Codeur.com adaptées à ton niveau.

APRÈS-MIDI — LE CADRE LÉGAL (SURVOL SÉRIEUX)

Pour facturer en France : la **micro-entreprise**. Renseigne-toi sur : l'inscription (gratuite, en ligne via le guichet unique des formalités sur procedures.inpi.fr), les cotisations sociales (un pourcentage du chiffre d'affaires encaissé, autour du quart pour les prestations de services — **vérifie les taux en vigueur sur urssaf.fr**, ils évoluent), la franchise de TVA sous les seuils, les mentions obligatoires sur devis et factures. Tu peux t'inscrire dès maintenant ou attendre ton premier « oui » ferme — mais sache que l'immatriculation prend quelques jours. *(Ceci est un repérage, pas un conseil juridique ou comptable : vérifie les chiffres officiels du moment.)*

OBJECTIF MESURABLE

5 messages envoyés, 2 candidatures déposées, et je sais concrètement comment je facturerai mon premier client. **À partir du Jour 43 : 30 min de prospection par jour, tous les jours** (5 nouveaux contacts ou relances). C'est une habitude, pas un événement.

AUTO-CONTRÔLE DE FIN DE SEMAINE 6

- Défi technique : ajouter une page « Avis clients » au site restaurant (route, lien navbar, contenu depuis un tableau d'objets) en 30 minutes.
- Récite ton pitch en 3 lignes sans le lire. S'il ne tient pas en 20 secondes à l'oral, on le retravaille.
- Montre-moi ta liste de 20 prospects et les 5 messages envoyés. Pas envoyés = jour non validé, sans exception.

SEMAINE 7 · JOURS 43–49

Backend : Node, Express, MongoDB — tu deviens full-stack

Jusqu'ici tes apps vivaient dans le navigateur. Cette semaine tu construis l'autre moitié : un serveur, une API, une vraie base de données. Nouveau rythme quotidien : 2 h 30 apprentissage · 2 h 30 pratique · 30 min consolidation · **30 min prospection (non négociable)**.

JOUR 43 Node.js : JavaScript côté serveur

CONCEPTS

Ce qu'est Node (JS hors navigateur), différences avec le navigateur (pas de DOM, accès aux fichiers) · modules ES (`"type": "module"` dans package.json) · modules natifs `fs` et `path` (survol) · variables d'environnement avec `process.env` et le package **dotenv** · scripts npm.

RESSOURCES

Doc Node.js, guide « Introduction to Node.js » (nodejs.org/learn — anglais simple, bon entraînement) · Grafikart, formation **« Apprendre NodeJS »** (premiers chapitres, FR).

EXERCICE

Script Node qui lit un fichier `produits.json` et génère un `rapport.txt` (nombre de produits, valeur totale du stock) · deuxième script qui affiche un message différent selon une variable d'environnement chargée depuis un fichier `.env` (et mets `.env` dans le .gitignore — réflexe vital).

OBJECTIF MESURABLE

J'explique ce qu'est Node, je lance mes scripts avec npm run, et je sais pourquoi .env ne va JAMAIS sur GitHub.

JOUR 44 Express : ton premier serveur

CONCEPTS

Ce qu'est un serveur HTTP · Express : `app.get`, `app.post`, objet `req` (params, query, body) et `res` (`res.json`, `res.status`) · middleware : le concept + `express.json()` · relance auto avec `node --watch` (ou nodemon) · tester sans navigateur avec **Thunder Client** (extension VS Code) ou Postman.

RESSOURCES

Doc Express (expressjs.com — le guide « Hello world » et « Basic routing » existent en français) · Grafikart NodeJS (chapitres Express).

EXERCICE

API « citations » : `GET /citations` (toutes), `GET /citations/:id` (une seule, 404 si absente), `POST /citations` (ajout, données en mémoire dans un tableau). Teste chaque route dans Thunder Client, y compris les cas d'erreur.

OBJECTIF MESURABLE

Je crée un serveur Express avec 3 routes et je le teste méthodiquement sans navigateur.

JOUR 45 API REST : concevoir proprement

CONCEPTS

Principes REST : ressources, verbes HTTP (GET / POST / PUT / DELETE) · **codes de statut** et quand les utiliser : 200, 201, 400, 404, 500 · organisation du code : séparer routes et logique (controllers) · validation basique des entrées (champ manquant → 400 avec message clair) · le concept de CORS (on le réglera en pratique au Jour 48).

RESSOURCES

MDN : « Méthodes de requête HTTP » et « Codes de statut de réponse HTTP » (FR — fiches de référence à garder sous le coude) · doc Express (routing, survol du chapitre middleware).

EXERCICE

API CRUD complète **« menu du restaurant »** (toujours en mémoire) : GET all / GET one / POST / PUT / DELETE, avec les bons codes de statut et une validation d'entrée sur le POST et le PUT. C'est le squelette que tu brancheras sur une vraie base demain.

OBJECTIF MESURABLE

Je conçois une API REST cohérente et je justifie chaque code de statut renvoyé.

JOUR 46 MongoDB : ta première base de données

CONCEPTS

Panorama honnête SQL vs NoSQL (tu feras du SQL au mois 4 — un pro connaît les deux) · MongoDB : documents, collections · créer un cluster gratuit sur **MongoDB Atlas** (cloud) · explorer avec **Compass** (interface graphique) · CRUD à la main dans Compass.

RESSOURCES

Doc MongoDB Atlas « Get Started » (guidée pas à pas) · MongoDB University propose des mini-cours gratuits (anglais) si tu veux approfondir · attention à bien noter ton utilisateur/mot de passe de base et à autoriser ton IP.

EXERCICE

Crée le cluster, une base `restaurant`, une collection `plats` · insère 8 plats à la main dans Compass · entraîne-toi aux requêtes de filtre dans Compass (plats d'une catégorie, prix inférieur à X).

OBJECTIF MESURABLE

J'ai une base de données dans le cloud, opérationnelle, et je sais y lire/écrire manuellement.

JOUR 47 Mongoose : brancher l'API sur la base

CONCEPTS

Ce qu'est un ODM · connexion à Atlas (URI dans `.env` !) · `Schema` et `Model`, validations de schéma (required, min…) · CRUD asynchrone : `find`, `findById`, `create`, `findByIdAndUpdate`, `findByIdAndDelete` · gestion d'erreurs avec try/catch → 500.

RESSOURCES

Doc Mongoose « Getting Started » puis « Queries » (mongoosejs.com, anglais clair) · Grafikart NodeJS (chapitres base de données).

EXERCICE

Remplace le tableau en mémoire de ton API menu par MongoDB via Mongoose : les 5 routes CRUD persistent réellement. Vérifie dans Compass que chaque opération modifie bien la base. Ajoute la validation de schéma (nom requis, prix positif).

OBJECTIF MESURABLE

Mon API survit à un redémarrage : les données sont réellement stockées. Je comprends le trajet requête → Express → Mongoose → Atlas → réponse.

JOUR 48 Full-stack : relier React à ton API

CONCEPTS

Architecture front/back séparés (deux dossiers, deux serveurs en dev) · **CORS** en pratique : l'erreur célèbre et le package `cors` · fetch depuis React vers `localhost:3000` · formulaire React → POST vers l'API · rafraîchir la liste après ajout/suppression.

RESSOURCES

MDN « CORS » (lire l'intro pour comprendre le pourquoi) · doc du package cors sur npm · tes patterns React de la semaine 5.

EXERCICE

Mini interface d'administration du menu en React : liste des plats (GET), formulaire d'ajout (POST), bouton supprimer (DELETE), états loading/error. Front et back tournent en même temps sur ta machine et se parlent.

OBJECTIF MESURABLE

Mon front React et mon back Express dialoguent en local, et je sais expliquer ce qu'est CORS à voix haute.

JOUR 49 Déploiement full-stack : tout en production

CONCEPTS

Déployer l'API sur **Render** (plan gratuit, connexion GitHub) : commande de démarrage, **variables d'environnement en production** (l'URI Mongo se met dans le dashboard Render, jamais dans le code) · autoriser les connexions dans Atlas (Network Access) · déployer le front sur Vercel avec l'URL de l'API en variable d'environnement Vite (`VITE_API_URL`) · notion de build.

RESSOURCES

Doc Render « Deploy a Node Express App » · doc Vercel (variables d'environnement) · doc Vite (variables d'env côté client).

EXERCICE

Mets toute la chaîne en production : API sur Render + front sur Vercel + base sur Atlas. Teste de bout en bout depuis ton téléphone. Dessine sur papier le schéma complet de l'architecture (navigateur → Vercel → Render → Atlas) : tu le redessineras à un client un jour.

OBJECTIF MESURABLE

Une application full-stack 100 % en ligne, gratuite, et je sais expliquer son architecture avec un schéma. Note : sur le plan gratuit Render, l'API s'endort après inactivité et met ~30 s à se réveiller — c'est normal, sache le dire.

AUTO-CONTRÔLE DE FIN DE SEMAINE 7

- Décris le trajet complet d'un clic « Ajouter un plat » jusqu'à la donnée écrite en base, étape par étape.
- Quand renvoyer 201 ? 400 ? 404 ? 500 ?
- Pourquoi l'URI MongoDB ne doit-elle jamais apparaître sur GitHub, et où la met-on en production ?
- Défi : ajouter une route `GET /plats/categorie/:cat` qui filtre côté base — 25 minutes.
- Prospection : combien de contacts cette semaine ? Moins de 25 (5/jour) = on en parle sérieusement.

SEMAINE 8 · JOURS 50–56

Authentification + PROJET FINAL full-stack

Dernière grande brique technique : savoir gérer des comptes utilisateurs proprement. Puis 4 jours pour LE projet qui prouve que tu es full-stack — celui que tu montreras à chaque prospect. Rythme : 2 h 30 / 2 h 30 / 30 min conso / 30 min prospection.

JOUR 50 Authentification : concepts + hachage des mots de passe

CONCEPTS

Pourquoi on ne stocke **jamais** un mot de passe en clair · hachage avec **bcrypt** (sel, coût) : hacher ≠ chiffrer · panorama sessions vs **JWT** (tu implémentes JWT, tu sais expliquer les deux) · anatomie d'un token (header.payload.signature) · où stocker le token côté client et les risques associés (survol honnête du sujet XSS).

RESSOURCES

Pages npm des packages `bcryptjs` et `jsonwebtoken` (les README suffisent) · jwt.io pour décoder un token et comprendre sa structure · Grafikart : cherche « JWT » ou « authentification » sur grafikart.fr pour un support vidéo FR.

EXERCICE

Sur ton API restaurant : modèle Mongoose `User` (email unique, password) · route `POST /auth/register` qui valide l'entrée, hache le mot de passe avec bcrypt et crée l'utilisateur (409 si email déjà pris) · ouvre Compass et **constate de tes yeux** que le mot de passe stocké est illisible.

OBJECTIF MESURABLE

J'explique à voix haute pourquoi on hache, ce qu'est un sel, et la différence hachage / chiffrement.

JOUR 51 Login + routes protégées côté serveur

CONCEPTS

`POST /auth/login` : retrouver l'utilisateur, comparer avec `bcrypt.compare`, signer un JWT (payload minimal : id, secret dans `.env`, expiration) · **middleware d'authentification** : lire le header `Authorization: Bearer …`, vérifier le token, attacher l'utilisateur à `req` · différence **401** (pas authentifié) vs **403** (pas autorisé).

RESSOURCES

README de `jsonwebtoken` (sign / verify) · doc Express sur les middlewares · tes fiches codes HTTP du Jour 45.

EXERCICE

Protège les routes d'écriture du menu : POST / PUT / DELETE exigent un token valide, GET reste public. Dans Thunder Client, déroule tout le scénario : login → copie du token → requête avec le header → succès ; puis sans token → 401 ; token bidonné → 401.

OBJECTIF MESURABLE

Mon API distingue un visiteur d'un utilisateur connecté, et je sais dérouler le cycle login → token → requête protégée dans Thunder Client sans notes.

JOUR 52 Authentification côté React

CONCEPTS

Formulaires login / register en React · stocker le token (localStorage pour ce niveau, en connaissant les limites) · **useContext** : un contexte d'authentification (user, login(), logout()) accessible partout · routes protégées côté front (rediriger vers /login si pas de token) · envoyer le token dans les headers des fetch · gérer un 401 (déconnexion automatique).

RESSOURCES

fr.react.dev : « Passer des données en profondeur avec le contexte » · doc React Router : pattern de route protégée (composant qui vérifie puis `<Navigate>`).

EXERCICE

Branche ton interface d'admin du menu (Jour 48) : page /login → si succès, accès à /admin (liste + ajout + suppression de plats avec le token dans les headers) · bouton déconnexion · accès direct à /admin sans être connecté → redirection.

OBJECTIF MESURABLE

Le parcours complet inscription → connexion → action protégée → déconnexion fonctionne de bout en bout, front + back.

JOURS 53–56 PROJET 7 (final) — Application de réservation full-stack

LE BRIEF

« Chez Lucie » (ton univers de la semaine 6) veut gérer ses réservations en ligne. **Côté client** : consulter les créneaux disponibles, réserver (nom, email, date, heure, nombre de personnes), recevoir une confirmation à l'écran. **Côté admin (authentifié)** : tableau de bord des réservations (par date), confirmer / annuler, gérer les plats du menu. Stack complète : **React + Tailwind / Express / MongoDB / JWT**, le tout déployé (Vercel + Render + Atlas).

JOUR 53 — CONCEPTION + SOCLE

On ne code pas tout de suite : schéma des données sur papier (collections `reservations`, `plats`, `users` et leurs champs), liste exhaustive des endpoints avec leurs codes de statut, wireframe rapide des écrans. Puis : initialisation du back (Express + Mongoose + structure de dossiers) et du front (Vite + React + Tailwind + Router).

JOUR 54 — L'API COMPLÈTE

Toutes les routes : CRUD réservations (création publique, lecture/modification protégées), CRUD plats (lecture publique, écriture protégée), auth (register/login — l'admin est créé une fois). Validation sérieuse des entrées (dates cohérentes, nombre de personnes plausible). Tout testé dans Thunder Client avant d'écrire une ligne de front.

JOUR 55 — LES DEUX INTERFACES

Front client : page réservation avec formulaire validé et retour clair (succès / erreur). Front admin : login, dashboard des réservations groupées par date, boutons confirmer / annuler, gestion des plats. États loading / error partout.

JOUR 56 — FINITIONS + MISE EN LIGNE

Responsive complet, messages d'erreur humains, données de démo réalistes (script de seed), **compte admin de démonstration** (demo@… / mot de passe simple) pour tes prospects, déploiement complet, README en étude de cas (contexte → solution → stack → schéma d'architecture → captures), ajout au portfolio.

OBJECTIF MESURABLE

LE projet qui prouve « full-stack » : un client potentiel peut tester la réservation ET l'espace admin avec le compte de démo. C'est ton meilleur argument de vente pour les 10 prochains mois.

AUTO-CONTRÔLE DE FIN DE SEMAINE 8

- Dessine de mémoire le cycle complet d'authentification JWT (inscription, connexion, requête protégée).
- Pourquoi bcrypt plutôt qu'un stockage en clair ou un simple chiffrement réversible ?
- 401 vs 403 : donne un exemple concret de chaque dans TON application.
- Défi : ajouter un champ « demandes spéciales » à la réservation, de la base jusqu'au formulaire (modèle → API → front) — 45 minutes.

SEMAINE 9 · JOURS 57–60

Finitions, portfolio v2, prospection intensive

Quatre jours pour transformer 8 semaines de travail en machine à décrocher des missions. Plus rien de nouveau techniquement : on polit, on met en valeur, on vend.

JOUR 57 Audit qualité de TOUS les projets

PROGRAMME

Passe chaque projet (1 à 7) au crible de la checklist qualité en annexe : Lighthouse (performance, accessibilité, SEO), favicon, meta description, page 404, aucun lien mort, aucune faute d'orthographe, responsive vérifié sur ton téléphone, README uniforme avec captures d'écran propres. Corrige tout ce qui coince — c'est long et ingrat, c'est exactement pour ça que ça te démarquera.

OBJECTIF MESURABLE

Chaque URL de mon portfolio peut être envoyée à un client sans que j'aie à rougir de quoi que ce soit.

JOUR 58 Portfolio v2 : orienté conversion

PROGRAMME

Refonte du portfolio (garde-le en statique amélioré ou repasse-le en React — le contenu prime sur la techno) : **études de cas détaillées** pour les 3 meilleurs projets (problème du client → solution apportée → stack → résultat, captures + lien live + démo admin pour le projet 7) · **page « Services »** avec 2-3 offres packagées et fourchettes de prix (voir guide prospection) · courte section « comment je travaille » (étapes, délais) · appel à l'action unique et visible partout : « Demander un devis gratuit » · demande à 2-3 proches de tester le parcours et corrige ce qui les perd.

OBJECTIF MESURABLE

Mon portfolio n'est plus une galerie technique : un commerçant pressé comprend en 30 secondes ce que je vends, à quel prix indicatif, et comment me contacter.

JOUR 59 Prospection : passage à l'échelle

PROGRAMME

Structure ta machine commerciale : **tableau de suivi** (prospect, canal, date, relance prévue, statut) · séquence de relance **J+3 puis J+7** (poli, court, nouvelle accroche à chaque fois) · prépare tes réponses aux 3 objections classiques : « pas de budget » (proposer une version réduite), « mon neveu fait les sites » (fiabilité, délais, maintenance), « on verra plus tard » (proposer un mini-audit gratuit daté) · dépose **10 candidatures** soignées sur Codeur.com / Malt · publie ton **premier post LinkedIn** : ton parcours 60 jours avec captures avant/après — c'est un excellent aimant à premières missions.

OBJECTIF MESURABLE

30+ prospects contactés au total depuis le Jour 42, un système de relance en place, 10 candidatures déposées, un post publié.

JOUR 60 Bilan + passage en rythme de croisière

MATIN — BILAN HONNÊTE

Remplis la grille d'auto-évaluation en annexe, section par section. Note tes 3 points forts et tes 3 points faibles techniques : les faibles deviennent des priorités du mois 3. Puis on fait le point ensemble : envoie-moi « Jour 60 terminé » avec ta grille remplie — je te poserai une dernière série de questions de validation, plus costaude que d'habitude.

APRÈS-MIDI — LE NOUVEAU RYTHME

À partir du mois 3, tu passes en **rythme de croisière : 2 à 3 h/jour** — 1 h d'apprentissage (le thème du mois), 1 h à 1 h 30 de projet/mission, 30 min de prospection quotidienne **tant que tu n'as pas de mission en cours**. Quand une mission tombe : elle devient prioritaire absolue, l'apprentissage passe à 30 min/jour. Planifie tes créneaux de la semaine prochaine dès maintenant.

OBJECTIF MESURABLE

Je sais exactement où j'en suis, ce qui me manque, et à quoi ressemblent mes 30 prochains jours. La phase intensive est finie ; l'activité commence.

CE QUE TU DOIS ÊTRE CAPABLE DE FAIRE AU JOUR 60 (LE CONTRAT)

- Construire et mettre en ligne un site vitrine responsive et propre en quelques jours, seul.
- Construire une application React branchée sur une API que tu as toi-même développée (Express + MongoDB), avec authentification, déployée.
- Expliquer ton architecture à un non-technicien avec un schéma simple.
- Trouver, contacter et relancer des prospects avec un portfolio crédible et des offres claires.

Si un de ces quatre points n'est pas tenu, on le répare en priorité au mois 3 — sans se raconter d'histoires.

Mois 3 à 12 — plan de croisière

Rythme : **2 à 3 h par jour**. Chaque mois combine un **axe technique** (1 h/jour), un **livrable concret**, et la **prospection continue** (30 min/jour hors mission). Règle absolue : une mission client payée passe toujours avant le programme d'apprentissage — c'est elle, la vraie formation. Le détail jour par jour de chaque mois sera construit ensemble le moment venu, en fonction de tes missions réelles.

Mois 3 — TypeScript + premier contrat PRIORITÉ BUSINESS

Technique : TypeScript, les bases utiles (types, interfaces, typage des props React et des réponses d'API) — formation gratuite Grafikart « Apprendre TypeScript » + doc officielle. Convertis l'app recettes en TS pour t'exercer. **Business :** objectif **100 prospects contactés en cumulé** et **1 à 2 premières missions signées**, même modestes (300–600 €). Chaque livraison = demander un témoignage écrit + une mise en relation. C'est le mois où l'on accepte d'être un peu sous-payé pour construire la preuve sociale — pas gratuit pour autant.

Mois 4 — SQL : PostgreSQL + Prisma TECHNIQUE

Le SQL est incontournable en mission. Bases de SQL (SELECT, JOIN, agrégats) puis **PostgreSQL** (base gratuite sur Neon ou Supabase) et l'ORM **Prisma** côté Node. Livrable : **refais l'API du projet final en PostgreSQL + Prisma** — refaire un projet connu avec une techno nouvelle est le moyen le plus rapide de l'apprendre. Tu sauras désormais choisir et justifier SQL vs MongoDB selon le projet.

Mois 5 — Next.js TRÈS VENDEUR

Le framework React de référence pour les sites clients : rendu côté serveur, **SEO nettement meilleur** (argument commercial massif pour les commerces), routing fichiers, déploiement Vercel natif. Doc officielle nextjs.org (le tutoriel « Learn » est excellent) + Grafikart a du contenu Next en français. Livrable : **migre le site restaurant en Next.js** et mesure la différence Lighthouse/SEO — tu tiens un nouvel argument de vente chiffré.

Mois 6 — Paiement en ligne + emails TRÈS VENDEUR

Stripe en mode test (doc officielle remarquable) : paiement simple, puis lien de paiement / acompte. **Emails transactionnels** (confirmation de réservation, reçu) avec Resend ou Nodemailer. Livrable : ajoute au projet final le **versement d'un acompte à la réservation + email de confirmation**. « Je peux encaisser vos clients en ligne » ouvre une gamme de missions bien mieux payées.

Mois 7 — Tests + qualité professionnelle CRÉDIBILITÉ

Ce qui sépare l'amateur du pro : **Vitest** (tests unitaires), **React Testing Library** (tests de composants), **Supertest** (tests d'API), et une **CI GitHub Actions** simple qui lance les tests à chaque push. Livrable : couvre les fonctions critiques du projet final (calculs, auth, routes principales). Bonus prospection : « code testé » rassure les clients tech et les agences qui sous-traitent.

Mois 8 — CMS headless : sites administrables REVENU RÉCURRENT

Le vrai besoin des petits clients : **modifier leur site eux-mêmes**. Apprends un CMS headless (**Strapi**, français et open source, ou **Sanity**) branché sur ton front Next/React. Livrable : le site restaurant devient administrable (menu, photos, horaires modifiables par « Lucie »). Ouvre l'offre **maintenance mensuelle** (30–80 €/mois par site) : ton premier revenu récurrent.

Mois 9 — Serveurs et DevOps : ton mois Linux TECHNIQUE

Le moment idéal pour muscler ton Linux, avec un vrai objectif : louer un petit **VPS** (Hetzner ou OVH, quelques €/mois), s'y connecter en SSH, sécuriser les bases (utilisateur, pare-feu, mises à jour), installer **nginx** + certificat SSL (Let's Encrypt), et déployer ton projet final dessus. Introduction à **Docker** (conteneuriser ton API). Livrable : ton app tourne sur TON serveur — indépendance des plans gratuits et compétence différenciante en mission.

Mois 10 — React avancé + performance TECHNIQUE

Monte en gamme sur ton outil principal : **useReducer** et useContext avancé, **TanStack Query** (la vraie façon de gérer les données serveur : cache, revalidation), un state manager léger (**Zustand**), optimisation (memo, lazy loading, code splitting, images). Livrable : refactore le dashboard admin du projet final avec TanStack Query et mesure le gain. Tu commences à lire du code d'équipe sans être perdu — utile pour les missions en régie.

Mois 11 — Spécialisation + produit personnel DIFFÉRENCIATION

Après ~5 mois de terrain, tes missions t'ont montré où est TA demande. Choisis une spécialisation et creuse-la à fond : **e-commerce** (Stripe avancé, catalogues), **dashboards / outils métier** (SaaS-like), ou **sites premium + SEO** pour indépendants. Livrable : un **produit personnel ambitieux** dans cette niche, pensé comme vitrine commerciale (et pourquoi pas premier micro-SaaS). On définira le cahier des charges ensemble.

Mois 12 — Bilan d'année + changement de braquet BUSINESS

Audit complet : compétences, portfolio (3–5 **études de cas réelles** désormais), chiffre d'affaires, clients récurrents. Revalorisation : **TJM cible 300–400 €** si tu as livré régulièrement, offres packagées revues à la hausse, tri des clients (garder les récurrents sains). Plan de l'année 2 : soit scaler le freelancing (plus cher, moins de clients), soit pousser le produit du mois 11. On fera ce bilan ensemble, chiffres sur la table.

Fil rouge des 10 mois

Prospection 30 min/jour hors mission, sans exception · chaque mission livrée = témoignage + étude de cas + demande de recommandation · Codewars ou équivalent 2×/semaine pour garder la main · veille légère (newsletter ou chaîne YouTube dev FR : Grafikart, Underscore_, Benjamin Code) — 30 min/semaine, pas plus : la veille ne remplace pas la pratique.

Guide prospection freelance : clients, pitch, tarifs, statut

Ta référence commerciale permanente. Tu l'appliques dès le Jour 41, puis 30 minutes par jour jusqu'à ce que ton agenda soit plein. Le meilleur développeur sans clients gagne zéro euro ; un développeur correct avec une bonne machine commerciale vit très bien.

Où trouver tes premiers clients (par ordre d'efficacité pour un débutant)

Canal	Comment l'utiliser
Démarchage local direct	Ton meilleur canal au début. Google Maps : commerces bien notés sans site, ou avec un site daté / pas responsive. Message court et personnalisé : un constat précis sur LEUR présence en ligne + une proposition simple + ton portfolio. 5 contacts/jour.
Codeur.com	Plateforme française très active sur les petits projets (300–2 000 €). Parfaite pour débuter : réponds vite (les premières heures comptent), message personnalisé qui prouve que tu as lu l'annonce, lien direct vers le projet le plus proche du besoin.
Bouche-à-oreille	Annonce à tout ton entourage (famille, amis, anciens collègues) que tu crées des sites : un message simple + 2 exemples. Les 2-3 premiers clients viennent souvent de là.
Malt	La place de marché freelance française de référence. Profil complet indispensable (photo, titre orienté client, 3 projets illustrés, tarif). Peu de missions au tout début sans avis — les avis de tes premiers clients hors plateforme peuvent s'y refléter via la cooptation ; sois patient, ça paie à partir des premières recommandations.
LinkedIn	Profil optimisé + 1 post/semaine (avancées, avant/après, mini-conseils aux commerçants). Le post « ce que j'ai appris en 60 jours » du Jour 59 est ton lancement.
Réseau local physique	CCI de ton département (ateliers créateurs), meetups dev et entrepreneurs (meetup.com), associations de commerçants. Un « je crée des sites pour les commerces du coin » en face à face vaut 50 emails.

Le pitch : parle résultats, jamais technologies

Mauvais : « Développeur full-stack junior passionné, je maîtrise React, Node.js, Express et MongoDB. »

Bon : « J'aide les commerçants et indépendants à obtenir un site moderne qui leur amène des clients : visible sur Google, impeccable sur mobile, avec réservation ou devis en ligne. Exemples : [lien portfolio]. Premier échange et estimation gratuits. »

Le client achète plus de réservations, plus d'appels, plus de crédibilité. React et MongoDB sont tes outils, pas ses arguments. Garde la démo du projet 7 (avec compte admin de démo) pour l'appel : c'est elle qui conclut.

Grille tarifaire de départ (France, fourchettes indicatives)

Prestation	Fourchette débutant	Notes
Landing page one-page	300 – 800 €	2-4 jours de travail réel au début. Ton projet 1-2 en est la démo.
Site vitrine 3-6 pages	600 – 1 500 €	Ton offre cœur. Le site restaurant (projet 6) en est la démo.
Site + fonctionnalité (réservation, devis, espace admin)	1 500 – 3 000 €	Ton projet 7 en est la démo. Monte vers le haut de la fourchette dès 2-3 références.
Maintenance / évolutions	30 – 80 € / mois	Propose-la systématiquement à la livraison : revenu récurrent + client fidélisé.
Régie (TJM)	150 – 250 € / jour	Pour les missions type agence/startup via Malt. Vise 300–400 € au mois 12.

Règles d'argent

Jamais gratuit — sauf 1 ou 2 projets stratégiques choisis par TOI (une association, un commerçant très visible) contre témoignage écrit + droit d'en faire une étude de cas. **Toujours un acompte de 30-50 %** à la commande, jamais de livraison finale sans solde. **Toujours un périmètre écrit** (devis détaillé signé) : le « tu peux juste ajouter… » non facturé est la mort du freelance. Annonce un délai réaliste puis livre un jour en avance.

La routine quotidienne (30 min, dès le Jour 43)

- **10 min :** 2-3 nouveaux prospects identifiés et ajoutés au tableau de suivi (prospect, canal, date, statut, relance prévue).
- **15 min :** 3-5 messages envoyés — nouveaux contacts, candidatures Codeur/Malt, et surtout les **relances J+3 et J+7** (la majorité des « oui » arrivent à la relance, pas au premier message).
- **5 min :** mise à jour du tableau. Le vendredi : bilan chiffré de la semaine (contacts, réponses, rendez-vous).

Répondre aux objections classiques

Objection	Réponse
« Pas de budget »	Proposer une version réduite qui règle LE problème principal (ex. landing one-page à la place du site complet), payable en 2-3 fois.
« Mon neveu / un ami fait les sites »	Sans dénigrer : engagement écrit, délais tenus, maintenance, résultat mesurable (Google, mobile). « Si votre site actuel vous amène des clients, gardez-le ; sinon, je vous montre en 15 min ce qui coince. »
« On verra plus tard »	Proposer un mini-audit gratuit daté (« je vous envoie 3 points concrets à corriger d'ici vendredi ») : ça crée le rendez-vous suivant.
Silence radio	Relance J+3 (angle différent), relance J+7 (dernier message poli). Ensuite on passe au suivant — sans état d'âme.

Le cadre légal : micro-entreprise (repérage)

Inscription : gratuite, en ligne, via le guichet unique des formalités d'entreprises (procedures.inpi.fr). Compte quelques jours pour recevoir ton SIRET — anticipe dès ton premier « oui » ferme, tu ne peux pas facturer sans.

Cotisations sociales : un pourcentage de ton chiffre d'affaires encaissé (de l'ordre du quart pour les prestations de services), déclaré à l'URSSAF (mensuel ou trimestriel). **Vérifie les taux en vigueur sur urssaf.fr** et regarde si tu as droit à l'ACRE (taux réduit en début d'activité).

TVA : en dessous des seuils de franchise, tu factures sans TVA avec la mention « TVA non applicable, art. 293 B du CGI ». Vérifie les seuils actuels.

Devis et factures : mentions obligatoires (identité, SIRET, détail des prestations, prix, délais, conditions de règlement), numérotation continue des factures. Un outil gratuit type Henrri ou les modèles de ton banquier suffisent au début. Une assurance RC pro est fortement recommandée dès les premières missions.

Ceci est un repérage pratique, pas un conseil juridique, fiscal ou comptable : les taux et seuils évoluent — vérifie systématiquement les sources officielles (urssaf.fr, impots.gouv.fr, inpi.fr) au moment de t'inscrire.

Annexes

Annexe A — Checklist qualité avant de publier un projet

À dérouler sur chaque projet avant de l'ajouter au portfolio ou de le livrer à un client. Utilisée au Jour 7, au Jour 57, puis à chaque livraison.

Domaine	Vérifications
Responsive	Testé en mode iPhone ET sur ton vrai téléphone · aucun défilement horizontal · textes lisibles sans zoomer · zones cliquables assez grandes.
Qualité de page	Lighthouse > 90 en performance et accessibilité · images compressées (squoosh.app) et avec `alt` · favicon · `title` unique et `meta description` par page.
Contenu	Zéro faute d'orthographe (fais-toi relire) · aucun lien mort · aucun texte de remplissage type « lorem ipsum » · page 404 propre.
Formulaires	Validation avec messages d'erreur clairs · message de succès · testés avec des données absurdes (champs vides, email invalide, texte très long).
Code & dépôt	Noms clairs, zéro code mort, formaté Prettier · README : description, captures, stack, lien live · aucun secret (.env, clés) dans l'historique Git.
Full-stack (si concerné)	Codes de statut cohérents · routes d'écriture protégées · gestion des erreurs réseau côté front (loading / error) · données de démo réalistes · compte de démonstration fonctionnel.

Annexe B — Grille d'auto-évaluation du Jour 60

Note-toi honnêtement : **A** = je le fais seul sans hésiter · **B** = j'y arrive avec la doc · **C** = je bloque. Tout C devient priorité du mois 3. Envoie-moi la grille remplie avec « Jour 60 terminé ».

Domaine	Je suis capable de…
HTML / CSS	Intégrer une maquette responsive propre (flexbox, grid, media queries) sans framework.
JavaScript	Manipuler tableaux d'objets (map / filter / reduce), le DOM, les événements, écrire des fonctions propres.
Asynchrone	Consommer une API avec async/await, gérer chargement et erreurs.
React	Construire une app multi-pages (composants, props, state, useEffect, Router) stylée en Tailwind.
Backend	Créer une API REST Express + MongoDB avec les bons codes de statut et de la validation.
Auth	Mettre en place inscription / connexion / routes protégées (bcrypt + JWT), front et back.
Déploiement	Mettre en production un projet full-stack (Vercel + Render + Atlas) avec variables d'environnement.
Git	Travailler en branches, fusionner, résoudre un conflit, maintenir des dépôts propres.
Débogage	Lire une erreur, poser des breakpoints, isoler un bug méthodiquement.
Commercial	Pitcher mon offre, contacter et relancer des prospects, produire un devis cadré.

Le mot de la fin (de ton coach)

Ce document est dense parce que le projet est sérieux. Tu n'as pas besoin d'être brillant pour le réussir : tu as besoin d'être **régulier**. 6 heures par jour, un jour après l'autre, en validant chaque étape — c'est tout. Les jours où tu douteras (il y en aura, autour des semaines 3 et 5 en général), relis la vue d'ensemble page 4 et regarde le chemin déjà fait. Et souviens-toi du protocole : chaque soir, « Jour X terminé » dans notre conversation. Je vérifie, on corrige, on avance. Au travail.